

FinShare Backend — Remaining Changes Required

For Abhilash · Backend v5 → v6

CONTEXT: This document covers only what still needs to be done in the backend. It does NOT ask you to redo or rethink anything that is already working well. Abhilash, your v5 is solid — security, events, group roles, audit logs, pagination, split tracking are all done correctly. These are the remaining gaps.

Summary — What Still Needs to Be Done

There are 4 remaining items. Items 1 and 2 are new additions. Items 3 and 4 are small fixes to existing code.

#	Item	Type
1	Add CurrencyCode to expenses	NEW — Required for international use
2	Wire notification triggers in 3 controllers	GAP — NotificationService not called on key events
3	Add mark-all-read endpoint to NotificationController	GAP — Missing endpoint
4	Remove duplicate /api/Expense/personal/my route	CLEANUP — Identical duplicate of /personal

1. Add CurrencyCode to Expenses (New Feature)

WHY THIS IS NEEDED: Currently the Expense table stores only Amount (decimal) with no currency information. If a user in Dubai adds AED 500 and a user in London adds GBP 500, the system sees only 500 and 500 — it cannot distinguish them. For international use this causes incorrect totals. This is an additive change — it will not break any existing data or APIs.

1A. Database Change

Add one nullable column to the dbo.Expenses table with a default value of 'AED':

```
ALTER TABLE dbo.Expenses
    ADD CurrencyCode NVARCHAR(3) NULL
        CONSTRAINT DF_Expenses_CurrencyCode DEFAULT 'AED';

-- After adding, back-fill existing rows:
UPDATE dbo.Expenses SET CurrencyCode = 'AED' WHERE CurrencyCode IS NULL;

-- Then make it NOT NULL if you prefer strict data:
ALTER TABLE dbo.Expenses
    ALTER COLUMN CurrencyCode NVARCHAR(3) NOT NULL;
```

1B. Update Stored Procedures

Update the following stored procedures to include CurrencyCode:

```
-- sp_AddExpense:    Add @CurrencyCode NVARCHAR(3) = 'AED' parameter, INSERT it
-- sp_EditExpense:   Add @CurrencyCode NVARCHAR(3) = NULL parameter, UPDATE it
-- sp_GetExpensesByGroup:  SELECT CurrencyCode in the result set
-- sp_GetPersonalExpenses: SELECT CurrencyCode in the result set
-- sp_AddPersonalExpense: Add @CurrencyCode NVARCHAR(3) = 'AED' parameter, INSERT it
-- sp_EditPersonalExpense: Add @CurrencyCode NVARCHAR(3) = NULL parameter, UPDATE it
```

1C. Update C# Models

AddExpenseRequest.cs — add one optional field:

```
[StringLength(3)]
public string CurrencyCode { get; set; } = "AED";
// Defaults to AED so existing callers require no change
```

AddPersonalExpenseRequest.cs — same addition:

```
[StringLength(3)]
public string CurrencyCode { get; set; } = "AED";
```

EditExpenseRequest.cs — add optional field:

```
[StringLength(3)]
public string? CurrencyCode { get; set; }
// Nullable — if null, keep existing value in the stored procedure
```

EditPersonalExpenseRequest.cs — same as edit:

```
[StringLength(3)]
public string? CurrencyCode { get; set; }
```

ExpenseResponse.cs — add to the response:

```
public string CurrencyCode { get; set; } = "AED";
```

1D. Update Service and Repository Method Signatures

Pass CurrencyCode through the chain:

```
// IExpenseRepository / ExpenseRepository:
//   AddExpenseAsync(..., string currencyCode = "AED")
//   EditExpenseAsync(..., string? currencyCode = null)
//   AddPersonalExpenseAsync(..., string currencyCode = "AED")
//   UpdatePersonalExpenseAsync(..., string? currencyCode = null)

// IExpenseService / ExpenseService:
//   Same signatures — pass through to repository

// ExpenseController.cs:
//   Pass request.CurrencyCode in each service call
```

CURRENCY CODES: Common currency codes for your users: AED (UAE), INR (India), USD (United States), GBP (United Kingdom), EUR (Europe), SGD (Singapore), SAR (Saudi Arabia). The frontend will show a dropdown — no automatic conversion needed.

2. Wire Notification Triggers in Controllers (Gap Fix)

THE GAP: You built `INotificationService` with a `CreateAsync()` method, but it is never called from `ExpenseController`, `SettlementController`, or `GroupController`. So users never receive any in-app notifications. The fix is to inject `INotificationService` into these three controllers and call `CreateAsync()` at the right moments.

2A. ExpenseController.cs

Inject `INotificationService` and call `CreateAsync` after a successful expense is added:

```
// 1. Add to constructor:
private readonly INotificationService _notifications;

public ExpenseController(
    IExpenseService expenseService,
    IGroupService groupService,
    IAttachmentStorageService attachmentStorage,
    INotificationService notifications) // <-- ADD THIS
{
    _notifications = notifications;
    ...
}

// 2. In Add() action, after result.IsSuccess:
// Notify all group members except the person who added the expense
var members = await _groupService.GetGroupMembersAsync(request.GroupId);
foreach (var member in members.Data!.Where(m => m.MemberId != memberId))
{
    await _notifications.CreateAsync(
        member.MemberId,
        "New Expense Added",
        $"{createdBy} added '{request.Description}' - {request.Amount:N2}",
        "expense",
        $"/group/{request.GroupId}")
    };
}

// 3. In Delete() action, after result.IsSuccess:
// Similar pattern - notify group members that an expense was removed
```

2B. SettlementController.cs

```
// 1. Add INotificationService to constructor (same pattern as above)

// 2. In RecordSettlement() action, after result.IsSuccess:
// Notify the person who is receiving the payment (toMemberId)
await _notifications.CreateAsync(
    request.ToMemberId,
    "Settlement Received",
    $"{fromName} marked a payment of {request.Amount:N2} to you",
    "settlement",
    "/settlement")
};
```

2C. GroupController.cs

```
// 1. Add INotificationService to constructor (same pattern as above)
```

```
// 2. In AddMember() action, after result.IsSuccess:  
// Notify the newly added member  
await _notifications.CreateAsync(  
    request.MemberId,  
    "Added to Group",  
    $"You have been added to group '{groupName}'",  
    "group",  
    $"/group/{groupId}"  
);
```

REMINDER: INotificationService is already registered in Program.cs (AddScoped). You only need to add it to the constructor — no DI changes required.

3. Add Mark-All-Read to NotificationController (Gap Fix)

THE GAP: NotificationController currently has GET (list) and PUT /{id}/read (mark one as read). There is no endpoint to mark ALL notifications as read at once. The frontend needs this for the 'Mark all as read' button on the notifications page.

3A. Add to INotificationService / NotificationService

```
// In INotificationService.cs - add:
Task MarkAllReadAsync(long memberId);

// In NotificationService.cs - implement:
public Task MarkAllReadAsync(long memberId)
    => _repository.MarkAllReadAsync(memberId);
```

3B. Add to INotificationRepository / NotificationRepository

```
// In INotificationRepository.cs - add:
Task MarkAllReadAsync(long memberId);

// In NotificationRepository.cs - implement:
public async Task MarkAllReadAsync(long memberId)
{
    using var conn = new SqlConnection(_config.GetConnectionString("DefaultConnection"));
    await conn.ExecuteAsync(
        "UPDATE dbo.Notifications SET IsRead = 1, ModifiedDate = GETUTCDATE() WHERE MemberId = @memberId AND IsRead = 0",
        new { memberId }
    );
}
// Note: can also create a stored procedure sp_MarkAllNotificationsRead if preferred
```

3C. Add to NotificationController.cs

```
/// <summary>PUT /api/Notification/read-all - Mark all notifications as read for the
current user.</summary>
[HttpPut("read-all")]
public async Task<IActionResult> MarkAllRead()
{
    var memberId = User.GetMemberId();
    await _notifications.MarkAllReadAsync(memberId);
    return Ok(ApiResponse<object>.SuccessResponse(null, "All notifications marked as read"));
}
```

4. Remove Duplicate Route (Cleanup)

WHAT TO REMOVE: In `ExpenseController.cs`, the `GetPersonal()` action has two `[HttpGet]` attributes mapping to the exact same method. The `/personal/my` route is a leftover alias that was never removed. The frontend uses `/api/Expense/personal` — the `/my` variant is never called.

```
// ExpenseController.cs – current (remove the second attribute):

[HttpGet("personal")]
[HttpGet("personal/my")] // <-- DELETE THIS LINE ONLY
public async Task<IActionResult> GetPersonal([FromQuery] string? category)
{
    ...
}

// After fix:

[HttpGet("personal")]
public async Task<IActionResult> GetPersonal([FromQuery] string? category)
{
    ...
}
```

That is the only change needed. No logic, no tests, no stored procedures — just delete one line.

What Is Already Well-Built — No Changes Needed

For reference, here is a confirmation of what was reviewed and found to be complete and correct in v5:

Feature	Status
JWT authentication + refresh token rotation	✔ Complete
Rate limiting (login, register, forgot password)	✔ Complete
AES-256 IBAN encryption (BankDetailsService)	✔ Complete
Group role management (Admin / Member)	✔ Complete
SplitType tracking (Equal / Custom) in ExpenseResponse	✔ Complete
Audit log (AuditLogRepository + sp_WriteAuditLog + retention)	✔ Complete
Group Events — full CRUD (GroupEventsController)	✔ Complete
Transaction history with skip/take pagination	✔ Complete
Notifications list with take parameter	✔ Complete
Profile photo resize 200x200 (ImageSharp)	✔ Complete
Personal vs group data separation (GroupId NULL for personal)	✔ Complete
User / Member model — clean and correct by design	✔ Complete
Service layer (BankDetailsService, NotificationService, etc.)	✔ Complete